An Integrated AIOps Framework for Predictive Failure Detection and Autonomous Resource Optimization in CI/CD Pipelines

Tejas Jagtap[1*], Charulata Patil[1] and Vinod Pachghare[1]

[1*]Department of Computer Science and Engineering, COEP Technological University, Pune, Maharashtra, 411005, India.

*Corresponding author(s). E-mail(s): jagtapta24.comp@coeptech.ac.in;
Contributing authors: cjp.comp@coeptech.ac.in;
vkp.comp@coeptech.ac.in;

Abstract

Continuous Integration and Continuous Deployment (CI/CD) pipelines are fundamental to modern DevOps practices, enabling automated and rapid software delivery. However, increasing pipeline complexity, voluminous log generation, and dynamic infrastructure workloads make traditional reactive monitoring approaches inadequate to ensure reliability and resource efficiency. This paper presents an integrated Artificial Intelligence for IT Operations (AIOps) framework that unifies three machine learning components for comprehensive CI/CD pipeline optimization: (1) XGBoost-based build failure prediction using structured CI metadata, (2) TF-IDF and autoencoder-based anomaly detection for console log analysis, and (3) Long Short-Term Memory (LSTM)-based time-series forecasting for proactive CPU utilization prediction. These models are orchestrated through a FastAPI-based coordination layer that enables closed-loop predictive Kubernetes autoscaling, transforming reactive DevOps monitoring into predictive, self-optimizing CI/CD operations. Experimental evaluation on a synthetic CI/CD dataset demonstrates that the proposed framework achieves strong failure prediction, effective log anomaly discrimination, and reliable workload forecasting, validating the feasibility and effectiveness of integrated AIOps architectures for predictive CI/CD pipeline management and autonomous resource optimization through closed-loop orchestration.

Keywords: AIOps, CI/CD Pipeline, Failure Prediction, Log Anomaly Detection, Predictive Autoscaling

1 Introduction

Modern software development relies critically on Continuous Integration and Continuous Deployment (CI/CD) pipelines to automate build, test, and deployment workflows. While these pipelines enable rapid and reliable software delivery, their growing complexity presents significant operational challenges. Empirical studies of open-source CI systems reveal high rates of recurring failures stemming from dependency conflicts, environmental inconsistencies, and test regressions [1, 2]. Such failures generate substantial volumes of build metadata, console logs, and infrastructure metrics, making manual monitoring impractical and error-prone at scale.

Traditional CI/CD monitoring mechanisms rely predominantly on static threshold-based alerting and post-hoc failure analysis. The Kubernetes Horizontal Pod Autoscaler (HPA) [3], for instance, adjusts container resources only after utilization thresholds are exceeded. Similarly, CI servers such as Jenkins detect build failures only after execution completes, resulting in wasted compute resources and delayed developer feedback. These reactive approaches are fundamentally inadequate for dynamic environments where workload patterns and failure modes evolve continuously [4].

Artificial Intelligence for IT Operations (AIOps) has emerged as a paradigm that integrates machine learning into operational workflows [5] to address these limitations. AIOps systems analyze heterogeneous operational signals logs, metrics, and metadata to provide predictive insights and enable autonomous remediation [6]. Recent advances have extended AIOps capabilities through agentic frameworks [7] and trustworthy production-grade ML pipelines [8], demonstrating growing maturity of the field. Applied to CI/CD pipelines, AIOps can enable proactive failure prediction, real-time anomaly detection, and predictive resource optimization, thereby transforming DevOps from reactive automation to intelligent orchestration.

Prior research has explored individual components of this transformation. Gradient boosting models such as XGBoost [9] have demonstrated effectiveness in classifying structured operational data. Reconstruction-based anomaly detection using autoencoders [10] and sequence modeling approaches [11, 12] have shown strong performance in identifying abnormal patterns within system logs. More recently, transformer-based architectures have been applied to log anomaly detection with promising results [13, 14]. LSTM networks [15] have been applied to forecast infrastructure workload for scaling decisions, with recent studies demonstrating their effectiveness in Kubernetes-specific deployments [16, 17]. However, these techniques have been studied predominantly in isolation, without integrated operational architectures.

A critical research gap lies in the absence of a unified framework that combines failure prediction, log anomaly detection, and predictive autoscaling within a single CI/CD ecosystem. Most existing studies address either autoscaling [18, 19] or anomaly detection [4, 20] independently, without establishing closed-loop architectures connecting predictions to autonomous operational decisions. As recent comprehensive reviews have noted [7, 8], the integration of multiple ML components into production-ready operational systems remains an open challenge.

This paper addresses this gap by proposing an integrated AIOps framework for CI/CD pipeline optimization. The framework combines XGBoost-based failure prediction for early identification of unstable builds, TF-IDF and autoencoder-based

log anomaly detection using reconstruction error, LSTM-based CPU forecasting for proactive Kubernetes autoscaling, and a FastAPI-based orchestration layer enabling closed-loop predictive scaling.

The principal contributions of this work are as follows. First, it presents the design of a unified AIOps architecture integrating supervised, unsupervised, and time-series models within CI/CD workflows, addressing the fragmentation identified in existing literature. Second, it implements and evaluates predictive build failure classification using gradient boosting with comprehensive feature importance analysis. Third, it develops a reconstruction-based anomaly detection mechanism for CI/CD console log streams using TF-IDF vectorization and autoencoder networks. Fourth, it designs and evaluates a predictive autoscaling strategy that leverages LSTM-based CPU forecasting for proactive resource management, surpassing the reactive capabilities of standard Kubernetes HPA. Fifth, it implements a modular FastAPI-based orchestration layer that forms a closed-loop architecture connecting ML predictions to autonomous Kubernetes scaling actions.

The remainder of the paper is organized as follows. Section 2 reviews background and related work. Section 3 presents the proposed framework architecture. Section 4 describes the experimental setup. Section 5 evaluates performance across all components. Section 6 provides a detailed discussion including comparative analysis, challenges, and limitations. Section 7 concludes the paper with directions for future research.

2 Background and Related Work

2.1 CI/CD Pipeline Reliability

Modern CI/CD pipelines automate the build-test-deploy cycle, enabling rapid software delivery in agile development environments. However, empirical studies demonstrate that large-scale CI systems frequently experience recurring failures, configuration drift, and performance degradation. Rausch et al. [1] analyzed build failures across open-source Java projects and identified that environmental and dependency-related failures constitute a significant proportion of CI breakages. Beller et al. [2] conducted an exploratory analysis of Travis CI with GitHub, finding that test failures are the primary cause of broken builds, with substantial variation across project types. Such failures increase mean time to recovery (MTTR), waste compute resources, and delay deployments. As pipeline complexity grows with microservices adoption and containerized workloads, reactive monitoring mechanisms become insufficient, motivating the integration of predictive analytics into CI/CD operational workflows. Recent surveys on AI-driven DevOps practices [4] have reinforced the urgency of this integration.

2.2 AIOps for Operational Intelligence

AIOps refers to the application of machine learning and data analytics to IT operations management. Dang et al. [5] outlined key AIOps challenges including operational data heterogeneity, alert fatigue, and the need for closed-loop automated remediation. Notaro et al. [6] conducted a systematic mapping study of AIOps research, identifying

anomaly detection, root cause analysis, and automated remediation as the principal pillars of AIOps practice. More recently, Zota et al. [7] proposed a practical framework for developing agentic AIOps systems, highlighting the evolution from reactive to proactive and ultimately autonomous IT operations. Bayram and Ahmed [8] provided a comprehensive overview of robustness challenges in production ML systems, emphasizing the importance of addressing concept drift, data quality, and model monitoring in operational deployments. While these studies establish theoretical and practical foundations, implementations integrating multiple ML components within CI/CD-specific workflows remain limited.

2.3 Build Failure Prediction

Predicting CI build outcomes from structured metadata has been explored using supervised learning. Gradient boosting frameworks, particularly XGBoost [9], have demonstrated strong performance due to their ability to model nonlinear feature interactions while incorporating regularization to mitigate overfitting. Ensemble methods including Random Forests [21] have also been applied to build outcome classification, though boosted models generally achieve superior accuracy on tabular feature sets. XGBoost formulates additive tree learning through regularized objective optimization, with the prediction for instance i given by:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F} \tag{1}$$

where f_k represents the kth regression tree, $\mathcal{F}$ is the space of Classification and Regression Trees (CART), and K is the number of boosting rounds. The regularized objective function is:

$$\mathcal{L} = \sum_{i=1}^{N} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{2}$$

where l denotes the loss function and $\Omega(f_k) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ is the regularization term with T leaf nodes and leaf weights w. The key advantages of XGBoost for build failure prediction include its handling of missing values, built-in regularization, and computational efficiency through column block structure and cache-aware access [9]. However, most failure prediction studies treat classification as a standalone task without connecting predictions to automated operational responses, limiting their practical impact in production CI/CD environments.

2.4 Log Anomaly Detection

CI/CD systems produce extensive console logs that contain critical diagnostic information. Automated log analysis has become a key research area, with early work by Xu et al. [22] establishing pattern-based log mining for large-scale system diagnosis. He et al. [23] provided a comprehensive empirical evaluation of multiple anomaly detection approaches, comparing principal component analysis, clustering, and supervised methods. Deep learning has advanced the field substantially. Du et al. [11] proposed

DeepLog, which models log sequences using recurrent neural networks to detect execution deviations. Meng et al. [12] introduced LogAnomaly, combining semantic and sequential information for unsupervised detection. Le and Zhang [20] benchmarked deep learning approaches for log anomaly detection, demonstrating that model architecture, training data quality, and threshold calibration significantly impact detection performance. Recent advances have explored transformer-based approaches. Liao and Liu [13] proposed combining transformers with temporal convolutional networks for log anomaly detection, achieving improved multi-dimensional feature extraction. Zhou et al. [14] leveraged large language models and BERT for log parsing and anomaly detection, demonstrating that pre-trained language representations can capture semantic nuances in log data more effectively than traditional vectorization methods. Reconstruction-based methods using autoencoders [10] learn compact representations of normal behavior and identify anomalies through elevated reconstruction error. Alternative unsupervised methods such as Isolation Forest [24] operate through isolation-based mechanisms but may be less effective in high-dimensional feature spaces typical of TF-IDF log vectorization. The autoencoder approach offers advantages in computational efficiency and interpretability through reconstruction error scoring, making it suitable for integration into real-time CI/CD monitoring pipelines.

2.5 Predictive Autoscaling

Cloud-native CI/CD pipelines rely on container orchestration platforms such as Kubernetes [25] for resource management. The Kubernetes HPA adjusts resources reactively based on observed metrics [3], but this approach introduces scaling lag during workload fluctuations, leading to either over-provisioning or service degradation. Predictive autoscaling using time-series forecasting has been explored as a superior alternative. Lorido-Botran et al. [18] surveyed auto-scaling techniques for elastic cloud applications, identifying proactive model-based approaches as superior to reactive threshold methods. More recently, Guruge and Priyadarshana [16] demonstrated the effectiveness of hybrid Prophet-LSTM models for Kubernetes autoscaling, achieving significant improvements over reactive HPA. Da Silva et al. [17] proposed adaptive horizontal scaling using ANN-based load forecasting in Kubernetes clusters, confirming the viability of neural network approaches for proactive resource management. Machiraju et al. [19] provided a comprehensive taxonomy of ML-based autoscaling frameworks, establishing evaluation criteria for comparing proactive scaling approaches. LSTM networks [15] remain well-suited for modeling temporal dependencies in infrastructure metrics due to their gated memory architecture, which effectively captures both short-term fluctuations and long-term workload trends.

2.6 Research Gap

Existing literature treats failure prediction [1, 9], log anomaly detection [11, 13, 20], and predictive autoscaling [16, 18] as independent problems. As highlighted by recent comprehensive reviews and surveys [7, 8, 19], few studies integrate these components into a unified framework capable of predicting build failures before execution, detecting

anomalous log behavior during runtime, forecasting infrastructure workload for proactive scaling, and enabling closed-loop orchestration between prediction and action. This work addresses this gap through an end-to-end AIOps architecture combining supervised, unsupervised, and time-series models within a single CI/CD optimization framework, unified by a modular orchestration layer that connects predictions to autonomous operational decisions.

3 Methodology

3.1 AIOps Framework Architecture

The proposed architecture integrates failure prediction, log anomaly detection, and predictive autoscaling into a unified AIOps framework. The system comprises five interconnected layers: (i) a Data Collection Layer that ingests operational data from Jenkins CI and Prometheus monitoring systems, (ii) a Preprocessing and Feature Engineering Layer that transforms raw data into model-ready representations, (iii) a Machine Learning Layer hosting three specialized ML models for failure prediction, anomaly detection, and CPU forecasting, (iv) an AIOps Orchestration Layer that coordinates model outputs through a FastAPI-based service enabling unified decision-making, and (v) a Predictive Autoscaling Engine that executes Kubernetes scaling actions based on orchestrated predictions using the Kubernetes Python client. Figure 1 illustrates the end-to-end data flow from CI/CD pipelines and infrastructure monitoring to Kubernetes scaling actions. The architecture forms a closed feedback loop where scaling actions generate new metrics that feed subsequent prediction cycles.

3.2 Data Collection Layer

Two primary operational data sources are utilized:

3.2.1 Jenkins Build Metadata and Logs

Build duration, test statistics, failure counts, warning indicators, and console logs are extracted via the Jenkins REST API. Let the build dataset be represented as:

$$D = \{(x_i, y_i)\}_{i=1}^{N} \tag{3}$$

where $x_i \in \mathbb{R}^d$ denotes a structured build feature vector containing d engineered features, $y_i \in \{0, 1\}$ indicates build success (0) or failure (1), and N is the total number of builds. The feature vector encompasses eight dimensions: build duration, tests passed, tests failed, tests total, warning count, error count, failure count, and log length.

3.2.2 Prometheus CPU Metrics

Time-series CPU utilization metrics are collected at fixed intervals from the Prometheus monitoring system:

$$T = \{c_t\}_{t=1}^{M} \tag{4}$$

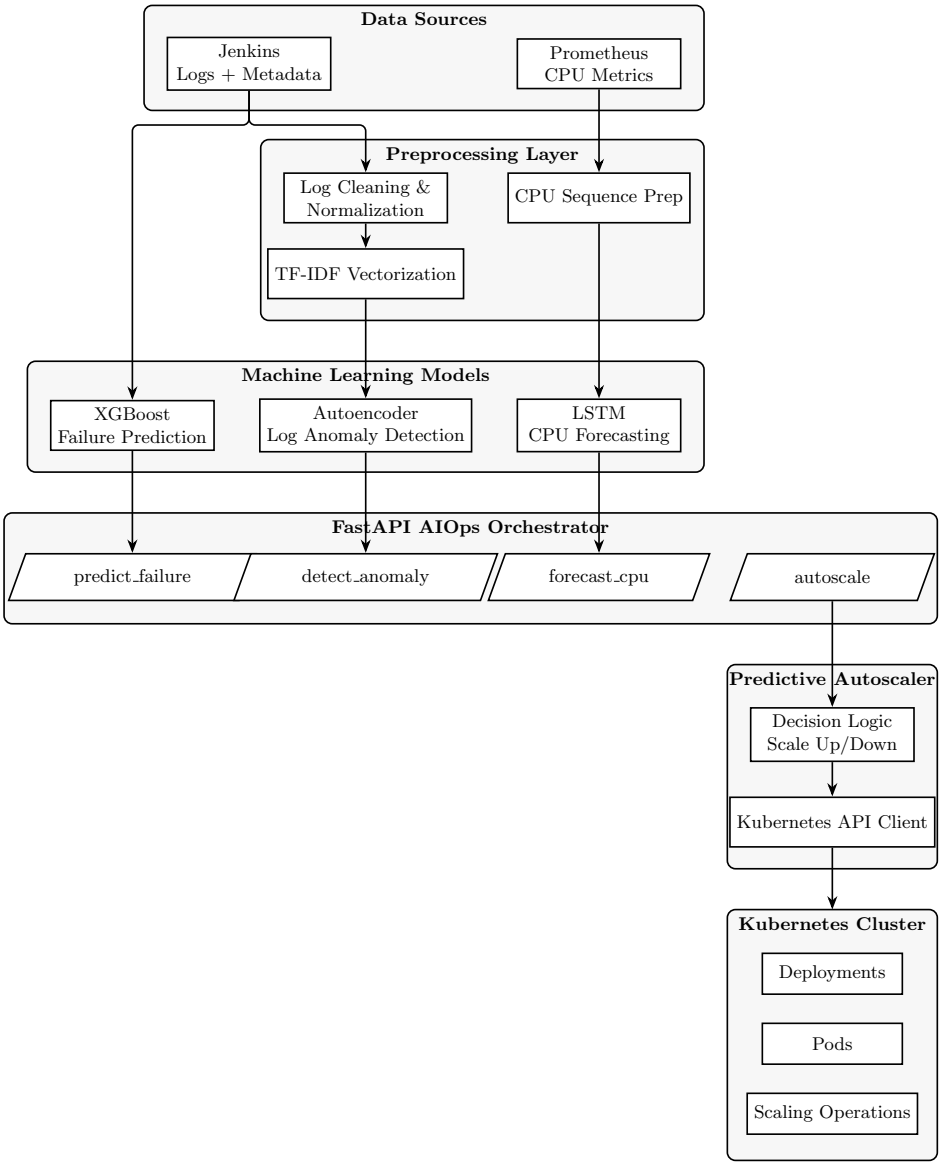


Fig. 1 Proposed AIOps framework architecture. Data flows from Jenkins CI and Prometheus monitoring through preprocessing and ML inference to the FastAPI orchestration layer, which coordinates predictive autoscaling decisions via Kubernetes.

where $c_t \in [0, 1]$ denotes normalized CPU utilization at time step t and M is the total number of observations. These metrics are collected at one-minute intervals, providing fine-grained temporal resolution for workload forecasting.

3.3 Failure Prediction Using XGBoost

Build outcome classification employs XGBoost [9], a gradient boosting framework that constructs an additive ensemble of decision trees. The prediction for build i is computed as:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F} \tag{5}$$

where f_k represents the kth regression tree and $\mathcal{F}$ is the space of CART trees. The objective function minimized during training is:

$$\mathcal{L} = \sum_{i=1}^{N} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{6}$$

where l is the logistic loss for binary classification and $\Omega(f_k) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ is the regularization term controlling model complexity through the number of leaf nodes T and leaf weight magnitude $\|w\|^2$. The hyperparameters γ and λ control the trade-off between model complexity and training loss. The model outputs a failure probability $P(y = 1|x) \in [0, 1]$, and a build is classified as high-risk when:

$$P(y = 1|x) > \tau_f \tag{7}$$

where τ_f is the failure probability threshold. Class imbalance is addressed using the `scale_pos_weight` parameter, which adjusts the gradient contribution of minority class samples. Feature importance is computed using the gain metric, which quantifies the average improvement in training loss when a feature is used in a split, enabling interpretable identification of the most influential failure predictors.

3.4 Log Anomaly Detection Using TF-IDF and Autoencoder

Console logs from Jenkins builds are analyzed through a two-stage pipeline combining TF-IDF feature extraction with autoencoder-based anomaly detection.

3.4.1 TF-IDF Feature Extraction

Raw console logs are first transformed into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right) \tag{8}$$

where $\text{TF}(t, d)$ is the term frequency of term t in document d, N is the total number of documents, and $\text{DF}(t)$ is the document frequency of term t. This transformation produces a sparse vector representation $z \in \mathbb{R}^n$ for each log, where n is the vocabulary size determined by the training corpus.

3.4.2 Autoencoder Architecture

The autoencoder learns compressed representations of normal log behavior through encoder and decoder mappings:

$$h = \sigma(W_e z + b_e), \quad \hat{z} = \sigma(W_d h + b_d) \tag{9}$$

where $h \in \mathbb{R}^m$ is the latent representation with $m \ll n$, $W_e \in \mathbb{R}^{m \times n}$ and $W_d \in \mathbb{R}^{n \times m}$ are the encoder and decoder weight matrices, b_e and b_d are bias vectors, and σ denotes the activation function (ReLU in hidden layers, Sigmoid in the output layer). The network architecture follows a symmetric encoder-decoder design with batch normalization and dropout ($p = 0.3$) regularization: Input(n) $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 (latent) $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ Output(n). The reconstruction loss is computed as the mean squared error:

$$\mathcal{L}_{\text{rec}} = \frac{1}{n} \|z - \hat{z}\|^2 \tag{10}$$

The model is trained exclusively on normal (non-anomalous) log samples, enabling it to learn the distribution of typical log patterns. An anomaly is flagged when:

$$\mathcal{L}_{\text{rec}} > \tau_a \tag{11}$$

where τ_a is the anomaly threshold determined from the 95th percentile of reconstruction errors computed on the normal validation set [10]. This threshold selection ensures high sensitivity to genuine anomalies while maintaining acceptable false positive rates.

3.5 CPU Forecasting Using LSTM

To enable proactive autoscaling, CPU utilization is modeled as a time-series forecasting task using an LSTM network [15]. Given a sliding window sequence of historical observations:

$$X_t = [c_{t-w}, c_{t-w+1}, \ldots, c_{t-1}] \tag{12}$$

where w is the lookback window size, the LSTM predicts future CPU usage $\hat{c}_t = f_\theta(X_t)$ through gated memory operations:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(Forget Gate)} \tag{13}$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(Input Gate)} \tag{14}$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad \text{(Candidate)} \tag{15}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(Cell State)} \tag{16}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(Output Gate)} \tag{17}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(Hidden Output)} \tag{18}$$

where f_t, i_t, o_t are the forget, input, and output gates respectively, C_t is the cell state, $\odot$ denotes element-wise multiplication, and σ is the sigmoid function. The gated architecture enables the network to selectively retain or discard temporal information, making it particularly effective for capturing both short-term fluctuations and long-term workload trends in CPU utilization data.

3.6 Orchestration and Predictive Autoscaling

A FastAPI-based orchestration layer integrates outputs from all three models through four REST API endpoints: `/predict_failure`, `/detect_anomaly`, `/forecast_cpu`, and `/autoscale`. The orchestration layer receives three key signals: failure probability $P(y = 1|x)$, anomaly score $\mathcal{L}_{\text{rec}}$, and predicted CPU utilization $\hat{c}_{t+1}$. Unlike the reactive Kubernetes HPA [3], which scales only after utilization thresholds are exceeded, the proposed system triggers scaling decisions using predicted values. The combined decision policy is formulated as:

$$S = \begin{cases} \text{Alert (Investigate)}, & \text{if } \mathcal{L}_{\text{rec}} > \tau_a \quad \text{(non-blocking)} \\ \text{Hold}, & \text{if } P(y = 1|x) > \tau_f \\ \text{Scale Up}, & \text{if } \hat{c}_{t+1} > \tau_c \\ \text{Scale Down}, & \text{if } \hat{c}_{t+1} < \tau_l \text{ and cooldown elapsed} \\ \text{Maintain}, & \text{otherwise} \end{cases} \tag{19}$$

where τ_c is the CPU scaling-up threshold, τ_l is the scaling-down threshold, and the cooldown period prevents oscillatory scaling behavior. The "Hold" action prevents resource scaling during builds with high failure probability, avoiding resource waste on likely-failing deployments. The "Investigate" action triggers alerts when log anomalies are detected, ensuring operational teams are informed of potential issues before they escalate. This policy ensures that scaling actions account not only for predicted resource demand but also for pipeline stability indicators. The system forms a closed-loop architecture (Fig. 1): data is collected from CI/CD and infrastructure sources, models generate predictions, the autoscaler adjusts resources through the Kubernetes Python client, and updated metrics feed back into subsequent prediction cycles. The autoscaling engine operates within configured boundaries including minimum and maximum replica counts and cooldown intervals, preventing both excessive resource consumption and service degradation. Algorithm 1 formalizes the complete autoscaling decision logic.

4 Experimental Setup

4.1 Dataset Generation

Due to the absence of publicly available CI/CD datasets containing synchronized build metadata, console log traces, and infrastructure metrics, a synthetic dataset was constructed using Python-based simulation scripts that emulate realistic CI/CD operational behavior. This approach enables controlled experimentation with known ground truth labels while preserving representative statistical properties observed in production environments [1, 2].

4.1.1 Build Metadata

A total of 600 Jenkins build records were generated, each containing eight structured features: build duration (seconds), tests passed, tests failed, tests total, warning count,

Algorithm 1 Predictive Autoscaling Decision Algorithm

Require: CPU forecast $\hat{c}_{t+1}$, failure probability P_f, anomaly score $\mathcal{L}_{\text{rec}}$
Require: Thresholds τ_c, τ_l, τ_f, τ_a; Replica limits $[r_{\min}, r_{\max}]$
Ensure: Scaling action S and target replicas r

$r_{\text{current}} \leftarrow$ GetCurrentReplicas
if $\mathcal{L}_{\text{rec}} > \tau_a$ **then**
    TriggerAlert("Log anomaly detected")
end if
if $P_f > \tau_f$ **then**
    $S \leftarrow$ Hold; $r \leftarrow r_{\text{current}}$ ▷ Avoid scaling for failing builds
else if $\hat{c}_{t+1} > \tau_c$ **and** $r_{\text{current}} < r_{\max}$ **then**
    $r \leftarrow \min(r_{\text{current}} + 1, r_{\max})$; $S \leftarrow$ ScaleUp
else if $\hat{c}_{t+1} < \tau_l$ **and** $r_{\text{current}} > r_{\min}$ **and** CooldownElapsed **then**
    $r \leftarrow \max(r_{\text{current}} - 1, r_{\min})$; $S \leftarrow$ ScaleDown
else
    $S \leftarrow$ Maintain; $r \leftarrow r_{\text{current}}$
end if
ApplyScaling(r)
return S, r

error count, failure count, and console log length (bytes). Build outcomes followed a realistic distribution: 68.3% SUCCESS, 22.8% FAILURE, and 8.8% UNSTABLE, consistent with empirical CI failure rates reported in the literature [1]. For binary classification, FAILURE builds were assigned label 1 and all others label 0. Failure characteristics were injected probabilistically based on elevated error counts, high test failure ratios, and abnormal build durations, ensuring that the synthetic data exhibits the correlational structure observed in real CI environments.

4.1.2 Console Log Data

A corpus of 600 Jenkins console log files was generated: 463 normal logs and 137 anomalous logs (22.8% anomaly rate). Normal logs follow consistent template patterns representing successful pipeline stages including source checkout, dependency installation, testing, build compilation, and artifact archiving. Anomalous logs contain injected patterns including unexpected stack traces, memory overflow errors (`java.lang.OutOfMemoryError`), timeout exceptions, dependency resolution failures, and segmentation faults. These anomaly patterns are drawn from commonly observed CI/CD failure modes documented in the literature [22, 23].

4.1.3 CPU Time-Series Data

A time-series of 10,080 data points was generated representing CPU utilization at one-minute intervals over a simulated seven-day period. The CPU signal is modeled as:

$$c_t = \mu + A_d \exp\left(-\frac{(h_t - h_p)^2}{2\sigma_d^2}\right) + A_w \exp\left(-\frac{(d_t - d_p)^2}{2\sigma_w^2}\right) + \alpha \frac{t}{M} + \epsilon_t + \delta_t \qquad (20)$$

where μ is baseline utilization, A_d is the diurnal amplitude with peak hour h_p and spread σ_d modeling workload variation during business hours, A_w is the weekly amplitude with peak day d_p modeling midweek workload increases, α represents a gradual upward trend over the observation period, $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise capturing measurement variability, and δ_t represents injected workload spike events occurring with fixed probability per time step with uniformly distributed magnitude. Table 1 summarizes the dataset characteristics.

Table 1 Dataset summary

Component	Characteristic	Value
Build Metadata	Total records	600
	Features	8
	Failure rate	22.8%
Console Logs	Total logs	600
	Normal / Anomalous	463 / 137
	Anomaly rate	22.8%
CPU Time-Series	Data points	10,080
	Sampling interval	1 minute
	Duration	7 days

4.2 Model Configuration

4.2.1 XGBoost Configuration

The failure prediction model was configured with the hyperparameters listed in Table 2. Class imbalance was addressed using the `scale_pos_weight` parameter computed as the ratio of negative to positive samples. Performance was validated using stratified 5-fold cross-validation to ensure consistent generalization across data folds.

Table 2 XGBoost hyperparameter configuration

Parameter	Value
Number of estimators (K)	100
Maximum tree depth	6
Learning rate (η)	0.1
Evaluation metric	Log loss
Class balancing	`scale_pos_weight`
Validation	Stratified 5-fold CV

4.2.2 Autoencoder Configuration

The anomaly detection autoencoder was configured with a symmetric architecture: Input $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 (latent) $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ Input. ReLU activation functions were used in hidden layers with batch normalization and dropout ($p = 0.3$) for regularization; a Sigmoid activation was applied at the output layer. Training was performed using the Adam optimizer [26] with a learning rate of 5×10^{-4}, batch size of 32, and 50 training epochs with early stopping (patience = 5 epochs). The model was trained exclusively on normal log samples to learn the distribution of typical log behavior. The anomaly threshold τ_a was set at the 95th percentile of reconstruction errors computed on the normal validation set.

4.2.3 LSTM Configuration

The CPU forecasting network consists of two stacked LSTM layers with 128 and 64 hidden units, with dropout ($p = 0.2$) between layers, followed by a fully connected layer with 32 units (ReLU activation) and a single Dense output unit with linear activation. A lookback window of $w = 60$ time steps and a forecast horizon of 10 time steps were used. Training was conducted for 50 epochs with the Adam optimizer, MSE loss function, batch size of 32, and early stopping (patience = 10 epochs) to prevent overfitting. A learning rate scheduler (ReduceLROnPlateau) was applied to adaptively reduce the learning rate during training. Input sequences were normalized using Min-Max scaling to the $[0, 1]$ range.

4.3 Data Partitioning

For all models, datasets were split into 80% training and 20% testing subsets. Stratified sampling was applied for classification tasks (failure prediction and anomaly detection) to preserve class distribution across splits. CPU time-series data was partitioned chronologically to avoid temporal data leakage, with the first 80% used for training and the remaining 20% further divided equally into validation and test sets, yielding approximately 1,000 test sequences.

4.4 Evaluation Metrics

Classification performance was evaluated using Accuracy, Precision, Recall, F1-Score, and Area Under the Receiver Operating Characteristic Curve (ROC-AUC). F1-Score was selected as the primary classification metric due to class imbalance, as it balances precision and recall.

Forecasting performance was assessed using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the coefficient of determination (R^2).

4.5 Implementation Details

The framework was implemented using Python 3.13 with the following key libraries: PyTorch 2.0 for deep learning models, scikit-learn 1.6 for preprocessing and evaluation, and XGBoost 2.1 for gradient boosting classification. The orchestration layer was

built using FastAPI 0.115 with Uvicorn as the ASGI server. Infrastructure monitoring employed Prometheus, and the container orchestration target was Kubernetes with scaling actions executed via the official Kubernetes Python client. All experiments were conducted on a system with an Intel Core i7 processor and 16 GB RAM running Windows 11.

5 Results and Performance Evaluation

5.1 Failure Prediction Results

The XGBoost classifier was evaluated on the held-out test set of 120 build records. Table 3 summarizes the classification performance. The model achieved an F1-Score of 0.982 with perfect recall (1.000), indicating that no failure cases were missed a critical requirement in CI/CD environments where undetected failures can propagate to production deployments, causing service disruptions and increasing MTTR. Stratified 5-fold cross-validation confirmed model stability, yielding a mean F1-Score of 0.975 ± 0.009 and mean ROC-AUC of 0.999 ± 0.001, demonstrating consistent generalization across data folds with minimal variance. The confusion matrix analysis shows only one false positive (stable build predicted as failure) and zero false negatives. The ROC curve demonstrates strong class separability with an AUC of 1.000. Feature importance analysis reveals that *tests_passed*, *tests_total*, and *error_count* are the most influential predictors, consistent with empirical CI failure analyses [1]. This finding provides actionable insights: CI/CD teams can prioritize monitoring these specific build attributes for early warning signals.

5.2 Log Anomaly Detection Results

The TF-IDF + Autoencoder model was evaluated on the held-out log test set of 120 samples. The model achieved F1-Score of 0.931 with perfect recall (1.000), successfully identifying all injected anomalies (Table 3). The reconstruction error distribution shows clear separation between normal and anomalous samples, validating the threshold-based detection strategy. The anomaly detection threshold τ_a, set at the 95th percentile of normal reconstruction errors, effectively delineates the two distributions. False positives (normal logs classified as anomalous) constitute the primary error mode, a consequence of the conservative threshold selection [10]. This configuration is operationally appropriate, as missed anomalies (false negatives) carry higher risk than false alerts in CI/CD environments where undetected anomalies can lead to cascading failures. The precision of 0.871 indicates that approximately 13% of anomaly alerts are false positives. While this rate is acceptable for operational use, it suggests room for improvement through more sophisticated feature engineering or adaptive threshold mechanisms.

5.3 CPU Forecasting Results

The LSTM model was evaluated on approximately 1,000 held-out test sequences derived from the chronologically partitioned CPU time-series. Table 4 presents the forecasting metrics.

The model achieved an RMSE of 0.070 and MAE of 0.043, indicating that predictions deviate from actual utilization by approximately 4.3 percentage points on average well within the operationally useful range for autoscaling decisions. The MAPE of 9.68% confirms consistent relative prediction accuracy across varying utilization levels. The R^2 value of 0.455 indicates the model captures a moderate proportion of variance, primarily reflecting the inherent stochasticity of the synthetic workload signal including injected Gaussian noise and randomly occurring spike events. The model effectively tracks dominant temporal trends (diurnal patterns) while smoothing high-frequency noise, which is the desired behavior for autoscaling rapid noise-driven scaling would introduce oscillatory behavior.

5.4 Consolidated Performance Summary

Table 3 Classification performance: Failure prediction and anomaly detection

Metric	XGBoost (Failure Pred.)	Autoencoder (Anomaly Det.)
Accuracy	0.992	0.967
Precision	0.964	0.871
Recall	1.000	1.000
F1-Score	0.982	0.931
ROC-AUC	1.000	1.000

Table 4 LSTM CPU forecasting performance

Metric	Value
MSE	0.0049
RMSE	0.0698
MAE	0.0433
MAPE	9.68%
R^2	0.455

5.5 Interpretation of Results

The experimental results demonstrate that the proposed AIOps framework achieves strong predictive performance across all three operational dimensions. Both classification models achieve perfect recall (1.000), prioritizing the detection of adverse events build failures and log anomalies at the cost of modest false positive rates. This design choice aligns with operational requirements in CI/CD environments, where missed failures carry substantially higher cost than false alerts [5]. The XGBoost failure predictor achieved near-perfect class separability (ROC-AUC = 1.000), driven by the

well-defined feature signatures in the dataset. The feature importance ranking with *tests_passed*, *tests_total*, and *error_count* as top predictors is consistent with empirical findings in CI/CD failure analysis [1, 2], providing external validity to the model's learned decision boundaries. The autoencoder-based anomaly detector achieved an ROC-AUC of 1.000, demonstrating effective learned representation of normal log behavior and sensitivity to injected anomaly patterns. The choice of TF-IDF vectorization, while effective, captures bag-of-words representations that may miss sequential dependencies in log data. Recent transformer-based approaches [13, 14] offer richer contextual embeddings but at significantly higher computational cost, presenting a trade-off relevant to real-time deployment scenarios. The LSTM forecasting module captures temporal workload trends sufficiently to support proactive scaling decisions. The moderate R^2 reflects the challenge of predicting stochastic workload components including sudden spikes, which is inherently bounded by the signal-to-noise ratio of the input data [16]. The RMSE of 0.070 represents an operationally useful level of prediction accuracy for threshold-based autoscaling decisions, comparable to results reported in recent Kubernetes autoscaling studies [17].

5.6 Comparative Context

Table 5 positions the proposed framework within the landscape of recent related work. Unlike studies that address individual components in isolation, the proposed framework provides an integrated architecture spanning all three operational dimensions.

Table 5 Comparison with related work

Study	Failure Pred.	Log Anomaly	Predictive Scaling	Closed-Loop
Du et al. [11]	×	✓	×	×
Le and Zhang [20]	×	✓	×	×
Liao and Liu [13]	×	✓	×	×
Guruge and Priyadarshana [16]	×	×	✓	×
Da Silva et al. [17]	×	×	✓	×
Kånåhols et al. [4]	×	✓	×	Partial
Zota et al. [7]	×	×	×	✓
Proposed Framework	✓	✓	✓	✓

Compared to traditional reactive monitoring, where failures and resource overloads are addressed only after occurrence, the proposed framework provides advance operational intelligence across build stability, log health, and resource demand. The integration of these signals through a unified orchestration layer represents a concrete step toward the closed-loop AIOps architectures advocated by Dang et al. [5], Notaro et al. [6], and more recently by Zota et al. [7].

6 Discussion

6.1 Practical Implications

The framework offers several practical benefits for DevOps teams. First, failure prediction enables pre-build risk assessment, allowing teams to prioritize code review or increase test coverage for high-risk commits before CI execution. Second, anomaly detection reduces the volume of logs requiring manual review by automatically flagging suspicious patterns, reducing alert fatigue [5]. Third, predictive autoscaling eliminates both over-provisioning (reducing cloud costs) and under-provisioning (preventing SLA violations) by anticipating resource demands. Fourth, the FastAPI-based architecture allows individual components to be deployed, updated, or replaced independently, supporting progressive adoption in existing CI/CD environments.

6.2 Synthetic Dataset Constraints

The evaluation was conducted on synthetically generated data. While controlled simulation enables reproducibility, structured experimentation, and known ground truth labels, it may not fully capture the complexity of production CI/CD environments, which exhibit noisy log streams, multi-service dependency failures, flaky tests, and infrastructure-level disruptions. The near-perfect ROC-AUC in failure prediction is partly attributable to the controlled separability of synthetic data. Production deployment would likely yield reduced performance due to noisier feature distributions and more complex failure modes. Validation on real-world CI/CD datasets from open-source projects (e.g., TravisTorrent, BuildFast) is an essential next step.

6.3 Concept Drift

CI/CD pipelines evolve through frequent codebase modifications, updated test suites, and infrastructure changes, introducing concept drift where statistical properties of input features shift over time [8, 27]. Models trained on historical data may degrade without periodic retraining. The current framework does not incorporate drift detection mechanisms. Future deployments must incorporate drift detection (e.g., ADWIN, Page-Hinkley test) and scheduled or incremental retraining strategies to maintain prediction accuracy over extended operational periods.

6.4 Scalability Considerations

Although the FastAPI orchestration layer provides modular integration, real-time inference at enterprise scale may introduce latency constraints. Organizations executing thousands of daily builds would require horizontally scaled model serving infrastructure with asynchronous inference capabilities (e.g., Triton Inference Server, TensorFlow Serving). Advanced log models leveraging transformer architectures [13, 28] may improve detection accuracy but impose significant computational overhead compared to the lightweight autoencoder approach employed here. The choice of model architecture should be guided by the specific latency and throughput requirements of the deployment environment.

6.5 Forecasting Limitations

The LSTM model effectively captured long-term workload trends but exhibited reduced sensitivity to abrupt spike events caused by batch merges, scheduled regression tests, or infrastructure incidents. Incorporating exogenous variables such as commit frequency, build queue length, deployment schedules, and calendar features (day-of-week, hour-of-day) may enhance forecasting robustness. Hybrid approaches combining LSTM with attention mechanisms or Prophet for multi-scale temporal modeling [16] represent promising directions.

6.6 Operational Integration Risks

Predictive autoscaling introduces decision complexity. Improperly calibrated thresholds may cause over-provisioning (increased cloud costs), under-provisioning (SLA violations), or resource oscillation (thrashing). The current threshold values (τ_c, τ_l, τ_f, τ_a) were determined empirically and may require environment-specific tuning. Careful policy tuning, validation through A/B testing, and gradual rollout with human-in-the-loop oversight are recommended before production deployment.

7 Conclusion and Future Work

This paper presented a unified AIOps framework for intelligent CI/CD pipeline optimization, integrating failure prediction, log anomaly detection, and predictive autoscaling within a single closed-loop operational architecture. The XGBoost failure predictor achieved an F1-Score of 0.982 with zero false negatives, ensuring comprehensive failure detection. The TF-IDF and autoencoder-based anomaly detector identified abnormal log patterns with an F1-Score of 0.931 and ROC-AUC of 1.000. The LSTM-based forecasting model captured temporal workload trends with an RMSE of 0.070 and MAE of 0.043, enabling proactive scaling that surpasses the reactive capabilities of standard Kubernetes HPA mechanisms [3].

The primary contribution lies in the comprehensive integration of supervised, unsupervised, and time-series models within a closed-loop orchestration layer. By combining failure signals, anomaly scores, and infrastructure forecasts through a unified decision policy, the framework enables coordinated operational decisions that improve reliability and resource efficiency. The modular FastAPI-based architecture ensures that individual components can be independently updated, replaced, or extended without disrupting the overall system.

Although evaluation was conducted on synthetic data, the results demonstrate the feasibility and potential impact of integrated AIOps strategies for CI/CD environments. The work advances the state of the art by providing a concrete implementation of the closed-loop AIOps architectures advocated in recent literature [5–7]. Future work will focus on deploying and evaluating the framework in production CI/CD pipelines with real operational data from open-source and enterprise environments, incorporating transformer-based log embeddings [13, 29] and large language model-based log parsing [14] for improved semantic understanding of console logs, implementing

concept drift detection and adaptive retraining pipelines [8, 27] to maintain prediction accuracy as CI/CD environments evolve, exploring reinforcement learning-based autoscaling policies that learn optimal scaling strategies through interaction with the deployment environment, and extending the framework to jointly optimize performance, cost, energy efficiency, and carbon footprint, contributing to sustainable DevOps practices.

Acknowledgements. The authors acknowledge the Department of Computer Science and Engineering, COEP Technological University, Pune, for providing the computational infrastructure and research environment for this work.

Declarations.

- **Funding:** Not applicable.
- **Conflict of interest:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** All authors consent to publication.
- **Data availability:** The synthetic datasets and source code used in this study are available from the corresponding author upon reasonable request.
- **Code availability:** The implementation source code including the orchestration layer, training scripts, and evaluation pipeline is available from the corresponding author upon reasonable request.
- **Author contribution:** T.J. conceptualized the framework, implemented the system, conducted experiments, and drafted the manuscript. C.P. contributed to methodology refinement and manuscript review. V.P. supervised the research and provided critical review.

References

[1] Rausch, T., Hummer, W., Leitner, P. & Schulte, S. An empirical analysis of build failures in the continuous integration workflows of java-based open-source software. *Proc. 14th IEEE/ACM Int. Conf. Min. Softw. Repos. (MSR)* 345–355 (2017).

[2] Beller, M., Gousios, G. & Zaidman, A. Oops, my tests broke the build: An explorative analysis of Travis CI with GitHub. *Proc. 14th IEEE/ACM Int. Conf. Min. Softw. Repos. (MSR)* 356–367 (2017).

[3] Kubernetes. Horizontal pod autoscaler. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (2024). Accessed: 2026-03-15.

[4] Kånåhols, G., Hasan, S. & Strandberg, P. E. Integrating time series anomaly detection into DevOps workflows. *IEEE Access* **13**, 46210–46225 (2025).

[5] Dang, Y., Lin, Q. & Huang, P. AIOps: Real-world challenges and research innovations. *Proc. 41st IEEE/ACM Int. Conf. Softw. Eng. Companion (ICSE-Companion)* 4–13 (2019).

[6] Notaro, P., Cardoso, J. & Gerndt, M. A systematic mapping study in AIOps. *Serv.-Oriented Comput. – ICSOC 2020 Workshops, Springer LNCS* 110–123 (2021).

[7] Zota, R. D., Bărbulescu, C. & Constantinescu, R. A practical approach to defining a framework for developing an agentic AIOps system. *Electronics* **14**, 1775 (2025).

[8] Bayram, F. & Ahmed, B. S. Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach. *ACM Computing Surveys* **57**, 1–38 (2025).

[9] Chen, T. & Guestrin, C. XGBoost: A scalable tree boosting system. *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.* 785–794 (2016).

[10] Sakurada, M. & Yairi, T. Anomaly detection using autoencoders with nonlinear dimensionality reduction. *Proc. 2nd Int. Workshop Mach. Learn. Sensory Data Anal. (MLSDA)* 4–11 (2014).

[11] Du, M., Li, F., Zheng, G. & Srikumar, V. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. *Proc. ACM Conf. Comput. Commun. Secur. (CCS)* 1285–1298 (2017).

[12] Meng, W. *et al.* LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. *Proc. 28th Int. Jt. Conf. Artif. Intell. (IJCAI)* 4739–4745 (2019).

[13] Liao, N. & Liu, Z. Log anomaly detection method based on transformer and temporal convolutional networks. *IEEE Access* **13**, 56489–56502 (2025).

[14] Zhou, Y. *et al.* Leveraging large language models and BERT for log parsing and anomaly detection. *Mathematics* **12**, 2758 (2024).

[15] Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Computation* **9**, 1735–1780 (1997).

[16] Guruge, P. B. & Priyadarshana, Y. Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and long short-term memory. *Frontiers in Computer Science* **7**, 1509165 (2025).

[17] Da Silva, L. M. D., Alves, P. V. A., Silva, S. N. & Fernandes, M. A. C. Adaptive horizontal scaling in Kubernetes clusters with ANN-based load forecasting. *Cluster Computing* **28**, 87 (2025).

[18] Lorido-Botran, T., Miguel-Alonso, J. & Lozano, J. A. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of Grid Computing* **12**, 559–592 (2014).

[19] Machiraju, V. S., Kumar, V. & Sharma, S. ML-based autoscaling for elastic cloud applications: Taxonomy, frameworks, and evaluation. *Mathematical and Computational Applications* **31**, 49 (2026).

[20] Le, V. H. & Zhang, H. Log-based anomaly detection with deep learning: How far are we? *Proc. 44th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)* 1356–1367 (2022).

[21] Breiman, L. Random forests. *Machine Learning* **45**, 5–32 (2001).

[22] Xu, W., Huang, L., Fox, A., Patterson, D. & Jordan, M. I. Detecting large-scale system problems by mining console logs. *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)* 117–132 (2009).

[23] He, P., Zhu, J., He, S., Li, J. & Lyu, M. R. An experience report on system log analysis for anomaly detection. *Proc. 27th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE)* 207–218 (2016).

[24] Liu, F. T., Ting, K. M. & Zhou, Z.-H. Isolation forest. *Proc. 8th IEEE Int. Conf. Data Min. (ICDM)* 413–422 (2008).

[25] Burns, B., Grant, B., Oppenheimer, D., Brewer, E. & Wilkes, J. Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue* **14**, 70–93 (2016).

[26] Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)* (2015).

[27] Abdullahi, M., Alhussian, H., Aziz, N. & Abdulkadir, S. J. A systematic literature review of concept drift mitigation in time-series applications. *IEEE Access* **13**, 78541–78560 (2025).

[28] Vaswani, A. *et al.* Attention is all you need. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **30**, 5998–6008 (2017).

[29] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proc. 2019 Conf. North Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. (NAACL-HLT)* 4171–4186 (2019).